Day 9 Task Workflow — Wednesday, Mar 5, 2026

> **Sprint:** 2 (Capture Engine + Core UI) | **Phase:** NSL-KDD Validation + ML Model Implementation Start
> **Owner:** Lead Architect | **Status:** ⚪ Ready to Start
> **Goal:** Validate NSL-KDD loader end-to-end, begin Isolation Forest implementation, prepare Week 2 demo
> **Grade:** Week 2 Day 2 A COMPLETE ☑ | Week 2 Day 3 STARTING ⚫

Day 9 Objective

Bridge Week 2 capture work into Week 3 ML pipeline by validating the NSL-KDD loader and beginning the first ML model so that by end of day:

- NSL-KDD loader validated: load, preprocess, split — with actual dataset files on VPS
- Feature mapping verified: live capture features align with NSL-KDD feature columns
- Isolation Forest model wrapper fully implemented (not just stub)
- Random Forest model wrapper fully implemented (not just stub)
- Model evaluation framework implemented with all required metrics
- Week 2 end-of-sprint verification complete (Sunday demo readiness check)

> **NOTE:** Frontend tasks remain with Full-Stack Dev per `docs/FRONTEND_TASKS_DAY8.md`. This document covers **Lead Architect tasks only.**

Scope Adherence Checklist

> **STRICT RULE:** Every task below MUST adhere to the master documentation specifications.

Requirement	Source Document	Section
NSL-KDD dataset loading + preprocessing	MASTER_DOC_PART4	§2.1, §2.4
Isolation Forest configuration	MASTER_DOC_PART4	§4.1-4.5
Random Forest configuration	MASTER_DOC_PART4	§5.1-5.5
Model evaluation metrics	MASTER_DOC_PART4	§7
Feature engineering validation	MASTER_DOC_PART4	§3.1-3.2
ML directory structure	MASTER_DOC_PART5	§2.1
Ensemble scoring weights	MASTER_DOC_PART4	§1.2

Architectural Constraints

> **ZERO TOLERANCE for deviation.**

Constraint	Rationale	Enforcement

Constraint	Rationale	Enforcement
scikit-learn IsolationForest	Stack locked per PART4 §4	Import from sklearn
scikit-learn RandomForestClassifier	Stack locked per PART4 §5	Import from sklearn
NSL-KDD 41 features + label	Academic requirement	Column validation
StandardScaler for normalization	PART4 §2.4	Preprocessing pipeline
LabelEncoder for categoricals	PART4 §2.4	protocol_type, service, flag
Stratified train/test split	PART4 §2.4	class balance preserved
joblib for model serialization	Standard practice	.pkl files in saved_models/
pandas for data manipulation	Stack locked	DataFrame operations

---

Task Breakdown

TASK 1 — NSL-KDD Loader Validation ⬛

Time Est: 60 min | **Priority:** ⬛ Critical

Validate `ml/datasets/nsl_kdd.py` end-to-end with the actual downloaded dataset files. This is the foundation for ALL model training.

1.1 Validation Script (`backend/ml/datasets/validate_nsl_kdd.py`)

Create a standalone validation script that tests the full loader pipeline:

```
"""
Validate NSL-KDD dataset loading and preprocessing.
Run on VPS: python -m ml.datasets.validate_nsl_kdd
"""

import logging
import sys
from pathlib import Path

import numpy as np

from ml.datasets.nsl_kdd import NSLKDDLoader, NSL_KDD_COLUMNS, ATTACK_CATEGORIES

logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s] %
(message)s")
logger = logging.getLogger(__name__)


def validate() -> bool:
    """Run full validation suite."""
    loader = NSLKDDLoader()
    errors = []
```

```
    # 1. Load raw data
    logger.info("=== Step 1: Load raw data ===")
    try:
        train_df = loader.load_train()
        test_df = loader.load_test()
        logger.info("Train: %d records, %d columns", len(train_df),
len(train_df.columns))
        logger.info("Test: %d records, %d columns", len(test_df),
len(test_df.columns))

        assert len(train_df) > 100000, f"Train too small: {len(train_df)}"
        assert len(test_df) > 20000, f"Test too small: {len(test_df)}"
        assert len(train_df.columns) == 43, f"Expected 43 columns, got
{len(train_df.columns)}"
    except Exception as e:
        errors.append(f"Load failed: {e}")
        logger.error("FAIL: %s", e)
        return False

    # 2. Check column names
    logger.info("=== Step 2: Verify column names ===")
    expected_cols = NSL_KDD_COLUMNS
    actual_cols = list(train_df.columns)
    if actual_cols != expected_cols:
        errors.append(f"Column mismatch: {set(expected_cols) - set(actual_cols)}")
    else:
        logger.info("PASS: All 43 columns match NSL-KDD spec")

    # 3. Check attack label distribution
    logger.info("=== Step 3: Attack label distribution ===")
    labels = train_df["label"].str.strip().str.lower()
    mapped = labels.map(ATTACK_CATEGORIES)
    unmapped = labels[mapped.isna()].unique()
    if len(unmapped) > 0:
        logger.warning("Unmapped labels: %s", unmapped)

    category_counts = mapped.dropna().value_counts()
    logger.info("Category distribution:\n%s", category_counts)

    assert "normal" in category_counts.index, "Missing 'normal' class"
    assert "dos" in category_counts.index, "Missing 'dos' class"
    assert "probe" in category_counts.index, "Missing 'probe' class"

    # 4. Preprocess
    logger.info("=== Step 4: Preprocess (fit) ===")
    X_train, y_train, feature_names = loader.preprocess(train_df, fit=True)
    logger.info("X_train shape: %s, y_train shape: %s", X_train.shape,
y_train.shape)
    logger.info("Feature names: %d features", len(feature_names))
    logger.info("Classes: %s", loader.get_class_names())

    assert X_train.shape[1] == len(feature_names), "Feature count mismatch"
    assert not np.isnan(X_train).any(), "NaN values in preprocessed data"
```

```
    assert not np.isinf(X_train).any(), "Inf values in preprocessed data"

    # 5. Preprocess test (transform only)
    logger.info("=== Step 5: Preprocess test set (transform) ===")
    X_test, y_test, _ = loader.preprocess(test_df, fit=False)
    logger.info("X_test shape: %s, y_test shape: %s", X_test.shape, y_test.shape)

    assert X_test.shape[1] == X_train.shape[1], "Train/test feature count
mismatch"

    # 6. Normal mask for unsupervised models
    logger.info("=== Step 6: Normal traffic mask ===")
    normal_mask = loader.get_normal_mask(y_train)
    normal_count = normal_mask.sum()
    logger.info("Normal samples: %d / %d (%.1f%%)",
                normal_count, len(y_train), 100 * normal_count / len(y_train))

    assert normal_count > 50000, f"Too few normal samples: {normal_count}"

    # 7. Summary
    logger.info("=" * 60)
    if errors:
        logger.error("VALIDATION FAILED: %d errors", len(errors))
        for e in errors:
            logger.error("  - %s", e)
        return False
    else:
        logger.info("✅ ALL VALIDATIONS PASSED")
        logger.info("  Train: %s → %s", train_df.shape, X_train.shape)
        logger.info("  Test:  %s → %s", test_df.shape, X_test.shape)
        logger.info("  Classes: %s", loader.get_class_names())
        logger.info("  Normal: %d samples (for unsupervised training)",
normal_count)
        return True


if __name__ == "__main__":
    success = validate()
    sys.exit(0 if success else 1)
```

1.2 Run on VPS

```
ssh root@187.124.45.161
cd /root/threatmatrix-ai
docker compose exec backend python -m ml.datasets.validate_nsl_kdd
```

Verification:

Check	Expected

Check	Expected
Train records loaded	125,973
Test records loaded	22,544
Column count	43 (41 features + label + difficulty)
All labels mapped	5 categories (normal, dos, probe, r2l, u2r)
X_train shape	(≈125K, 41)
No NaN/Inf	Clean data
Normal mask count	>50,000 samples

TASK 2 — Isolation Forest Implementation

Time Est: 90 min | **Priority:** Critical

Implement the full Isolation Forest model wrapper per MASTER_DOC_PART4 §4.

2.1 Implement `ml/models/isolation_forest.py`

```
"""
ThreatMatrix AI — Isolation Forest Model

Per MASTER_DOC_PART4 §4: Unsupervised anomaly detection.
Trained on NORMAL traffic only — anomalies are "few and different."

Hyperparameters from §4.3:
- n_estimators: 200
- contamination: 0.05
- max_samples: 'auto'
"""

from __future__ import annotations

import logging
from pathlib import Path
from typing import Any, Dict, Optional

import joblib
import numpy as np
from sklearn.ensemble import IsolationForest as SklearnIF

from ml.training.hyperparams import ISOLATION_FOREST_PARAMS

logger = logging.getLogger(__name__)

SAVE_DIR = Path(__file__).parent.parent / "saved_models"
```

```python
class IsolationForestModel:
    """
    Isolation Forest wrapper for ThreatMatrix AI.

    Training: Fit on normal traffic only (unsupervised).
    Inference: Score flows — higher score = more anomalous.
    """

    MODEL_NAME = "isolation_forest"

    def __init__(self, params: Optional[Dict[str, Any]] = None) -> None:
        self.params = params or ISOLATION_FOREST_PARAMS.copy()
        self.model: Optional[SklearnIF] = None
        self._is_fitted = False

    def train(self, X_normal: np.ndarray) -> Dict[str, Any]:
        """
        Train on normal traffic only.

        Args:
            X_normal: Feature matrix of NORMAL traffic samples only.

        Returns:
            Training metadata dict.
        """
        logger.info(
            "[IF] Training on %d normal samples with %d features",
            X_normal.shape[0], X_normal.shape[1]
        )

        self.model = SklearnIF(**self.params)
        self.model.fit(X_normal)
        self._is_fitted = True

        # Compute training scores for threshold calibration
        train_scores = self.model.decision_function(X_normal)
        train_preds = self.model.predict(X_normal)

        metadata = {
            "model": self.MODEL_NAME,
            "n_samples": X_normal.shape[0],
            "n_features": X_normal.shape[1],
            "params": self.params,
            "train_anomaly_rate": float((train_preds == -1).mean()),
            "train_score_mean": float(np.mean(train_scores)),
            "train_score_std": float(np.std(train_scores)),
        }

        logger.info(
            "[IF] Training complete. Anomaly rate on train: %.2f%%",
            metadata["train_anomaly_rate"] * 100
        )
        return metadata
```

```python
    def predict(self, X: np.ndarray) -> np.ndarray:
        """
        Predict anomaly labels.

        Returns:
            Array of 0 (normal) or 1 (anomaly).
        """
        if not self._is_fitted:
            raise RuntimeError("Model not trained. Call train() first.")

        raw_preds = self.model.predict(X)
        # sklearn returns 1=normal, -1=anomaly → convert to 0=normal, 1=anomaly
        return (raw_preds == -1).astype(int)

    def score(self, X: np.ndarray) -> np.ndarray:
        """
        Compute anomaly scores (0.0 = very normal, 1.0 = very anomalous).

        Returns:
            Normalized anomaly scores array.
        """
        if not self._is_fitted:
            raise RuntimeError("Model not trained. Call train() first.")

        # decision_function: lower = more anomalous
        raw_scores = self.model.decision_function(X)
        # Normalize to [0, 1] where 1 = most anomalous
        # Negate so higher = more anomalous, then min-max scale
        negated = -raw_scores
        min_val = negated.min()
        max_val = negated.max()
        if max_val - min_val == 0:
            return np.zeros(len(X))
        normalized = (negated - min_val) / (max_val - min_val)
        return normalized

    def save(self, path: Optional[Path] = None) -> Path:
        """Save trained model to disk."""
        if not self._is_fitted:
            raise RuntimeError("Cannot save untrained model.")

        save_path = path or SAVE_DIR / f"{self.MODEL_NAME}.pkl"
        save_path.parent.mkdir(parents=True, exist_ok=True)
        joblib.dump(self.model, save_path)
        logger.info("[IF] Model saved to %s", save_path)
        return save_path

    def load(self, path: Optional[Path] = None) -> None:
        """Load trained model from disk."""
        load_path = path or SAVE_DIR / f"{self.MODEL_NAME}.pkl"
        if not load_path.exists():
            raise FileNotFoundError(f"Model not found: {load_path}")

        self.model = joblib.load(load_path)
```

```
        self._is_fitted = True
        logger.info("[IF] Model loaded from %s", load_path)
```

Verification:

Check	Command	Expected
Import works	`python -c "from ml.models.isolation_forest import IsolationForestModel"`	No errors
Params match PART4	Check `self.params`	n_estimators=200, contamination=0.05
Train method	Mock data test	Returns metadata dict
Predict returns 0/1	Test predict	Binary array
Score returns 0-1	Test score	Normalized floats
Save creates .pkl	Call save()	File exists in saved_models/
Load restores model	Call load() then predict	Same predictions

TASK 3 — Random Forest Implementation 🟤

Time Est: 90 min | **Priority:** 🟤 Critical

Implement the full Random Forest classifier per MASTER_DOC_PART4 §5.

3.1 Implement `ml/models/random_forest.py`

```
"""
ThreatMatrix AI — Random Forest Classifier

Per MASTER_DOC_PART4 §5: Supervised multi-class classification.
Trained on ALL labeled data (normal + attacks).

Hyperparameters from §5.2:
- n_estimators: 300
- max_depth: 30
- class_weight: 'balanced'

Classes (NSL-KDD): normal, dos, probe, r2l, u2r
"""

from __future__ import annotations
```

```python
import logging
from pathlib import Path
from typing import Any, Dict, List, Optional

import joblib
import numpy as np
from sklearn.ensemble import RandomForestClassifier as SklearnRF

from ml.training.hyperparams import RANDOM_FOREST_PARAMS

logger = logging.getLogger(__name__)

SAVE_DIR = Path(__file__).parent.parent / "saved_models"


class RandomForestModel:
    """
    Random Forest wrapper for ThreatMatrix AI.

    Training: Fit on ALL labeled data (supervised classification).
    Inference: Predict class + confidence for each flow.
    """

    MODEL_NAME = "random_forest"

    def __init__(self, params: Optional[Dict[str, Any]] = None) -> None:
        self.params = params or RANDOM_FOREST_PARAMS.copy()
        self.model: Optional[SklearnRF] = None
        self.class_names: List[str] = []
        self._is_fitted = False

    def train(
        self, X: np.ndarray, y: np.ndarray, class_names: Optional[List[str]] =
None
    ) -> Dict[str, Any]:
        """
        Train on labeled data (all classes).

        Args:
            X: Feature matrix (all samples)
            y: Label array (integer encoded)
            class_names: Optional list of class names

        Returns:
            Training metadata dict.
        """
        self.class_names = class_names or [str(i) for i in
range(len(np.unique(y)))]

        logger.info(
            "[RF] Training on %d samples, %d features, %d classes",
            X.shape[0], X.shape[1], len(self.class_names)
        )
```

```
        self.model = SklearnRF(**self.params)
        self.model.fit(X, y)
        self._is_fitted = True

        # Training accuracy
        train_preds = self.model.predict(X)
        train_accuracy = float((train_preds == y).mean())

        # Feature importance
        importances = self.model.feature_importances_
        top_10_idx = np.argsort(importances)[::-1][:10]

        metadata = {
            "model": self.MODEL_NAME,
            "n_samples": X.shape[0],
            "n_features": X.shape[1],
            "n_classes": len(self.class_names),
            "class_names": self.class_names,
            "params": self.params,
            "train_accuracy": train_accuracy,
            "top_10_feature_indices": top_10_idx.tolist(),
            "top_10_importances": importances[top_10_idx].tolist(),
        }

        logger.info("[RF] Training complete. Train accuracy: %.4f",
train_accuracy)
        return metadata

    def predict(self, X: np.ndarray) -> np.ndarray:
        """Predict class labels."""
        if not self._is_fitted:
            raise RuntimeError("Model not trained. Call train() first.")
        return self.model.predict(X)

    def predict_proba(self, X: np.ndarray) -> np.ndarray:
        """Predict class probabilities."""
        if not self._is_fitted:
            raise RuntimeError("Model not trained. Call train() first.")
        return self.model.predict_proba(X)

    def predict_with_confidence(self, X: np.ndarray) -> List[Dict[str, Any]]:
        """
        Predict with label name and confidence.

        Returns:
            List of dicts with 'label', 'confidence', 'is_anomaly'.
        """
        predictions = self.predict(X)
        probabilities = self.predict_proba(X)

        results = []
        for i, (pred, proba) in enumerate(zip(predictions, probabilities)):
            label = self.class_names[pred] if pred < len(self.class_names) else
"unknown"
```

```
            confidence = float(np.max(proba))
            results.append({
                "label": label,
                "confidence": confidence,
                "is_anomaly": label != "normal",
                "class_probabilities": {
                    name: float(p) for name, p in zip(self.class_names, proba)
                },
            })
        return results

    def get_feature_importance(self, feature_names: List[str]) -> List[Dict[str,
float]]:
        """Get sorted feature importances."""
        if not self._is_fitted:
            raise RuntimeError("Model not trained.")
        importances = self.model.feature_importances_
        pairs = sorted(
            zip(feature_names, importances), key=lambda x: x[1], reverse=True
        )
        return [{"feature": name, "importance": float(imp)} for name, imp in
pairs]

    def save(self, path: Optional[Path] = None) -> Path:
        """Save trained model + class names."""
        if not self._is_fitted:
            raise RuntimeError("Cannot save untrained model.")
        save_path = path or SAVE_DIR / f"{self.MODEL_NAME}.pkl"
        save_path.parent.mkdir(parents=True, exist_ok=True)
        joblib.dump({"model": self.model, "class_names": self.class_names},
save_path)
        logger.info("[RF] Model saved to %s", save_path)
        return save_path

    def load(self, path: Optional[Path] = None) -> None:
        """Load trained model + class names."""
        load_path = path or SAVE_DIR / f"{self.MODEL_NAME}.pkl"
        if not load_path.exists():
            raise FileNotFoundError(f"Model not found: {load_path}")
        data = joblib.load(load_path)
        self.model = data["model"]
        self.class_names = data["class_names"]
        self._is_fitted = True
        logger.info("[RF] Model loaded from %s", load_path)
```

Verification:

Check	Command	Expected
Import works	`python -c "from ml.models.random_forest import RandomForestModel"`	No errors

Check	Command	Expected
Params match PART4 §5.2	Check `self.params`	n_estimators=300, max_depth=30, class_weight='balanced'
Train returns metadata	Mock data test	accuracy, feature importance
predict_with_confidence	Test output	label, confidence, is_anomaly
Feature importance	Call method	Sorted list of {feature, importance}
Save/load roundtrip	Save → load → predict	Identical results

TASK 4 — Model Evaluation Framework ⬛

Time Est: 60 min | **Priority:** ⬛ Critical

Implement `ml/training/evaluate.py` with all metrics per MASTER_DOC_PART4 §7.

4.1 Implement `ml/training/evaluate.py`

```
"""
ThreatMatrix AI — Model Evaluation Framework

Per MASTER_DOC_PART4 §7: Comprehensive evaluation metrics.

Computes: Accuracy, Precision, Recall, F1-Score, AUC-ROC,
Confusion Matrix, Classification Report, per-class metrics.
"""

from __future__ import annotations

import json
import logging
from pathlib import Path
from typing import Any, Dict, List, Optional

import numpy as np
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score,
    roc_auc_score,
)

logger = logging.getLogger(__name__)
```

```python
class ModelEvaluator:
    """
    Evaluate ML model performance with standard IDS metrics.

    Produces:
    - Overall accuracy, precision, recall, F1
    - Per-class metrics
    - Confusion matrix
    - AUC-ROC (multi-class, one-vs-rest)
    """

    def evaluate_binary(
        self,
        y_true: np.ndarray,
        y_pred: np.ndarray,
        y_scores: Optional[np.ndarray] = None,
        model_name: str = "model",
    ) -> Dict[str, Any]:
        """
        Evaluate a binary anomaly detection model (normal vs anomaly).

        Args:
            y_true: Ground truth (0=normal, 1=anomaly)
            y_pred: Predicted labels (0=normal, 1=anomaly)
            y_scores: Anomaly scores for AUC-ROC (optional)
            model_name: Name for logging
        """
        results: Dict[str, Any] = {
            "model": model_name,
            "type": "binary",
            "n_samples": len(y_true),
            "accuracy": float(accuracy_score(y_true, y_pred)),
            "precision": float(precision_score(y_true, y_pred, zero_division=0)),
            "recall": float(recall_score(y_true, y_pred, zero_division=0)),
            "f1_score": float(f1_score(y_true, y_pred, zero_division=0)),
            "confusion_matrix": confusion_matrix(y_true, y_pred).tolist(),
        }

        if y_scores is not None:
            try:
                results["auc_roc"] = float(roc_auc_score(y_true, y_scores))
            except ValueError:
                results["auc_roc"] = None

        logger.info(
            "[Eval] %s — Acc: %.4f | P: %.4f | R: %.4f | F1: %.4f",
            model_name,
            results["accuracy"],
            results["precision"],
            results["recall"],
            results["f1_score"],
        )
        return results
```

```
    def evaluate_multiclass(
        self,
        y_true: np.ndarray,
        y_pred: np.ndarray,
        y_proba: Optional[np.ndarray] = None,
        class_names: Optional[List[str]] = None,
        model_name: str = "model",
    ) -> Dict[str, Any]:
        """
        Evaluate a multi-class classification model.

        Args:
            y_true: Ground truth labels (integer encoded)
            y_pred: Predicted labels (integer encoded)
            y_proba: Predicted probabilities for AUC-ROC (optional)
            class_names: Class name list
            model_name: Name for logging
        """
        labels = sorted(np.unique(np.concatenate([y_true, y_pred])))

        results: Dict[str, Any] = {
            "model": model_name,
            "type": "multiclass",
            "n_samples": len(y_true),
            "n_classes": len(labels),
            "class_names": class_names or [str(l) for l in labels],
            "accuracy": float(accuracy_score(y_true, y_pred)),
            "precision_macro": float(precision_score(y_true, y_pred,
average="macro", zero_division=0)),
            "recall_macro": float(recall_score(y_true, y_pred, average="macro",
zero_division=0)),
            "f1_macro": float(f1_score(y_true, y_pred, average="macro",
zero_division=0)),
            "precision_weighted": float(precision_score(y_true, y_pred,
average="weighted", zero_division=0)),
            "recall_weighted": float(recall_score(y_true, y_pred,
average="weighted", zero_division=0)),
            "f1_weighted": float(f1_score(y_true, y_pred, average="weighted",
zero_division=0)),
            "confusion_matrix": confusion_matrix(y_true, y_pred,
labels=labels).tolist(),
            "classification_report": classification_report(
                y_true, y_pred,
                target_names=class_names or [str(l) for l in labels],
                output_dict=True,
                zero_division=0,
            ),
        }

        # AUC-ROC (one-vs-rest)
        if y_proba is not None:
            try:
                results["auc_roc_ovr"] = float(
```

```
                    roc_auc_score(y_true, y_proba, multi_class="ovr",
average="weighted")
                )
            except ValueError:
                results["auc_roc_ovr"] = None

        logger.info(
            "[Eval] %s — Acc: %.4f | F1(w): %.4f | F1(m): %.4f",
            model_name,
            results["accuracy"],
            results["f1_weighted"],
            results["f1_macro"],
        )
        return results

    def compare_models(self, results: List[Dict[str, Any]]) -> Dict[str, Any]:
        """Compare evaluation results across models."""
        comparison = {
            "models": [],
            "best_accuracy": None,
            "best_f1": None,
        }

        best_acc = 0.0
        best_f1 = 0.0

        for r in results:
            entry = {
                "model": r["model"],
                "accuracy": r["accuracy"],
                "f1": r.get("f1_score") or r.get("f1_weighted", 0),
            }
            comparison["models"].append(entry)

            if entry["accuracy"] > best_acc:
                best_acc = entry["accuracy"]
                comparison["best_accuracy"] = r["model"]
            if entry["f1"] > best_f1:
                best_f1 = entry["f1"]
                comparison["best_f1"] = r["model"]

        return comparison

    def save_results(
        self, results: Dict[str, Any], path: Optional[Path] = None
    ) -> Path:
        """Save evaluation results to JSON."""
        save_dir = Path(__file__).parent.parent / "saved_models" / "eval_results"
        save_dir.mkdir(parents=True, exist_ok=True)

        save_path = path or save_dir / f"{results['model']}_eval.json"
        with open(save_path, "w") as f:
            json.dump(results, f, indent=2, default=str)
```

```
        logger.info("[Eval] Results saved to %s", save_path)
        return save_path
```

Verification:

Check	Command	Expected
Import works	`python -c "from ml.training.evaluate import ModelEvaluator"`	No errors
Binary eval	Mock binary predictions	accuracy, precision, recall, f1, confusion_matrix
Multiclass eval	Mock multi-class predictions	All metrics + classification_report
AUC-ROC compute	With probability scores	Float 0-1
Compare models	Pass 2+ result dicts	best_accuracy, best_f1 identified
Save to JSON	Call save_results	JSON file created

TASK 5 — Training Orchestrator 🟢

Time Est: 60 min | **Priority:** 🟢 Medium

Implement `ml/training/train_all.py` — the master training script that trains IF + RF, evaluates, and saves everything.

5.1 Implement `ml/training/train_all.py`

```
"""
ThreatMatrix AI — Master Training Script

Trains all models on NSL-KDD dataset:
1. Load and preprocess NSL-KDD
2. Train Isolation Forest on normal data
3. Train Random Forest on all labeled data
4. Evaluate both models
5. Save models + evaluation results

Run: python -m ml.training.train_all
"""

from __future__ import annotations

import logging
import sys
```

```
import time
from pathlib import Path

import numpy as np

from ml.datasets.nsl_kdd import NSLKDDLoader
from ml.models.isolation_forest import IsolationForestModel
from ml.models.random_forest import RandomForestModel
from ml.training.evaluate import ModelEvaluator

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)
logger = logging.getLogger(__name__)


def train_all() -> bool:
    """Train all models and evaluate."""
    start_time = time.time()
    evaluator = ModelEvaluator()
    all_results = []

    # ── Step 1: Load NSL-KDD ──────────────────────────────────────
    logger.info("=" * 60)
    logger.info("STEP 1: Loading NSL-KDD dataset")
    logger.info("=" * 60)

    loader = NSLKDDLoader()
    train_df = loader.load_train()
    test_df = loader.load_test()

    X_train, y_train, feature_names = loader.preprocess(train_df, fit=True)
    X_test, y_test, _ = loader.preprocess(test_df, fit=False)

    normal_mask_train = loader.get_normal_mask(y_train)
    normal_mask_test = loader.get_normal_mask(y_test)
    class_names = loader.get_class_names()

    logger.info("Train: %s, Test: %s, Classes: %s", X_train.shape, X_test.shape,
class_names)

    # ── Step 2: Train Isolation Forest ────────────────────────────
    logger.info("=" * 60)
    logger.info("STEP 2: Training Isolation Forest (unsupervised)")
    logger.info("=" * 60)

    X_train_normal = X_train[normal_mask_train]
    logger.info("Normal training samples: %d", len(X_train_normal))

    if_model = IsolationForestModel()
    if_meta = if_model.train(X_train_normal)
```

```
    # Evaluate IF (binary: normal vs anomaly)
    y_test_binary = (~normal_mask_test).astype(int)  # 0=normal, 1=anomaly
    if_preds = if_model.predict(X_test)
    if_scores = if_model.score(X_test)

    if_eval = evaluator.evaluate_binary(
        y_true=y_test_binary,
        y_pred=if_preds,
        y_scores=if_scores,
        model_name="isolation_forest",
    )
    evaluator.save_results(if_eval)
    all_results.append(if_eval)

    if_model.save()
    logger.info("[IF] Accuracy: %.4f | F1: %.4f", if_eval["accuracy"],
if_eval["f1_score"])

    # ── Step 3: Train Random Forest ──────────────────────────────
    logger.info("=" * 60)
    logger.info("STEP 3: Training Random Forest (supervised, 5-class)")
    logger.info("=" * 60)

    rf_model = RandomForestModel()
    rf_meta = rf_model.train(X_train, y_train, class_names=class_names)

    # Evaluate RF (multi-class)
    rf_preds = rf_model.predict(X_test)
    rf_proba = rf_model.predict_proba(X_test)

    rf_eval = evaluator.evaluate_multiclass(
        y_true=y_test,
        y_pred=rf_preds,
        y_proba=rf_proba,
        class_names=class_names,
        model_name="random_forest",
    )
    evaluator.save_results(rf_eval)
    all_results.append(rf_eval)

    rf_model.save()

    # Feature importance report
    importance = rf_model.get_feature_importance(feature_names)
    logger.info("Top 10 features:")
    for i, fi in enumerate(importance[:10], 1):
        logger.info("  %d. %s: %.4f", i, fi["feature"], fi["importance"])

    # ── Step 4: Model Comparison ─────────────────────────────────
    logger.info("=" * 60)
    logger.info("STEP 4: Model Comparison")
    logger.info("=" * 60)

    comparison = evaluator.compare_models(all_results)
```

```
    logger.info("Best accuracy: %s", comparison["best_accuracy"])
    logger.info("Best F1: %s", comparison["best_f1"])

    # ── Summary ──────────────────────────────────────────────────
    elapsed = time.time() - start_time
    logger.info("=" * 60)
    logger.info("TRAINING COMPLETE in %.1f seconds", elapsed)
    logger.info("  Isolation Forest: saved to saved_models/isolation_forest.pkl")
    logger.info("  Random Forest:    saved to saved_models/random_forest.pkl")
    logger.info("  Evaluations:      saved to saved_models/eval_results/")
    logger.info("=" * 60)

    return True


if __name__ == "__main__":
    success = train_all()
    sys.exit(0 if success else 1)
```

Verification:

Check	Command	Expected
Script runs	`python -m ml.training.train_all`	Completes without errors
IF model saved	`ls saved_models/isolation_forest.pkl`	File exists
RF model saved	`ls saved_models/random_forest.pkl`	File exists
Eval results saved	`ls saved_models/eval_results/`	2 JSON files
IF accuracy	Check eval JSON	≥ 85%
RF accuracy	Check eval JSON	≥ 95%
Feature importance	Check logs	Top 10 printed

TASK 6 — Week 2 Demo Readiness Check 🟢

Time Est: 30 min | **Priority:** 🟢 Medium

Verify all Week 2 deliverables are ready for the Sunday demo per MASTER_DOC_PART5 §3.

6.1 End-of-Week 2 Verification

```
ssh root@187.124.45.161

# 1. All Docker services healthy
docker compose ps

# 2. Capture engine running with 63+ features
docker compose logs capture --tail=5
```

```
docker compose exec postgres psql -U threatmatrix -d threatmatrix \
  -c "SELECT jsonb_object_keys(features) FROM network_flows LIMIT 1;" \
  | wc -l

# 3. Flow count growing
docker compose exec postgres psql -U threatmatrix -d threatmatrix \
  -c "SELECT COUNT(*) FROM network_flows;"

# 4. NSL-KDD dataset present
ls -la backend/ml/saved_models/datasets/KDD*.txt

# 5. ML module structure complete
find backend/ml -type f -name "*.py" | wc -l

# 6. API health
curl -s http://localhost:8000/api/v1/system/health | python3 -m json.tool
curl -s http://localhost:8000/api/v1/capture/status | python3 -m json.tool

# 7. NSL-KDD loader validates
docker compose exec backend python -m ml.datasets.validate_nsl_kdd

# 8. No Docker version warning
docker compose config 2>&1 | grep -i version
```

Per MASTER_DOC_PART5 §3:

> **End of Week 2 Demo:** Live traffic being captured and stored. War Room layout visible with placeholder/mock data.

Requirement	Status	Evidence
Live traffic captured	☑	1,860+ flows in PostgreSQL
Traffic stored	☑	PostgreSQL network_flows table
63 features per flow	☑	JSONB in database
War Room layout visible	📋 Full-Stack Dev	Check FRONTEND_TASKS_DAY8.md
Capture API working	☑	/capture/status returns data
ML scaffold ready for Week 3	☑	18 files in backend/ml/

Files Created/Modified Today

```
threatmatrix-ai/
├── backend/
│   └── ml/
│       ├── datasets/
│       │   └── validate_nsl_kdd.py     🆕 TASK 1
│       ├── models/
│       │   ├── isolation_forest.py     🔨 TASK 2 (full implementation)
```

```
│       │   └── random_forest.py       🔨 TASK 3 (full implementation)
│       ├── training/
│       │   ├── evaluate.py            🔨 TASK 4 (full implementation)
│       │   └── train_all.py           🔨 TASK 5 (full implementation)
│       └── saved_models/
│           └── eval_results/          🆕 TASK 4 (output directory)
└── docs/
    └── worklog/
        └── DAY_09_MAR05.md            🆕 This file
```

Verification Checklist

> **Every item below MUST be verified before marking task complete.**

#	Verification	Expected Result
1	NSL-KDD train loads	125,973 records, 43 columns
2	NSL-KDD test loads	22,544 records, 43 columns
3	All labels mapped	5 classes (normal, dos, probe, r2l, u2r)
4	Preprocessing clean	No NaN/Inf values
5	Normal mask correct	>50,000 normal samples
6	IF import works	No errors
7	IF train completes	Returns metadata dict
8	IF predict binary	Returns 0/1 array
9	IF score normalized	Returns 0-1 float array
10	IF save/load roundtrip	Identical predictions
11	RF import works	No errors
12	RF train completes	Returns metadata with accuracy
13	RF predict gives classes	5-class integer array
14	RF predict_with_confidence	Label + confidence dicts
15	RF feature importance	Sorted list
16	RF save/load roundtrip	Identical predictions
17	Evaluator binary	Acc, P, R, F1, confusion matrix
18	Evaluator multiclass	Acc, F1 macro/weighted, AUC-ROC
19	Evaluator compare	Best model identified
20	train_all.py runs E2E	Both models trained + saved

#	Verification	Expected Result
21	Docker services healthy	4 running + ml-worker restarting
22	Flow count growing	> 1,860 in PostgreSQL
23	Python type hints	All functions typed
24	Error handling	Try/except, RuntimeError for untrained

Scope Adherence Verification

Requirement	Source	Verification
scikit-learn IsolationForest	MASTER_DOC_PART4 §4	sklearn.ensemble used
IF hyperparams from spec	MASTER_DOC_PART4 §4.3	n_estimators=200, contamination=0.05
IF trained on normal only	MASTER_DOC_PART4 §4.3	X_train[normal_mask]
scikit-learn RandomForest	MASTER_DOC_PART4 §5	sklearn.ensemble used
RF hyperparams from spec	MASTER_DOC_PART4 §5.2	n_estimators=300, class_weight='balanced'
RF trained on all classes	MASTER_DOC_PART4 §5.2	Full X_train, y_train
NSL-KDD 5 classes	MASTER_DOC_PART4 §5.3	normal, dos, probe, r2l, u2r
Evaluation metrics per spec	MASTER_DOC_PART4 §7	Acc, P, R, F1, AUC, CM
Feature importance	MASTER_DOC_PART4 §5.4	Top-10 printed
Model serialization	MASTER_DOC_PART5 §2.1	.pkl in saved_models/
StandardScaler used	MASTER_DOC_PART4 §2.4	In NSLKDDLoader
Stratified split	MASTER_DOC_PART4 §2.4	NSL-KDD has pre-split files

Blockers

Blocker	Severity	Mitigation	Status
Autoencoder (TensorFlow)	⚪ Medium	Day 10 task — separate due to TF complexity	Scheduled
ml-worker restarting	⚪ Medium	Expected until inference worker implemented (Week 4)	Expected
VPS training time unknown	⚪ Medium	KVM 4 has 4 vCPU — RF may take 5-10 min	Monitor
NSL-KDD class imbalance (R2L/U2R rare)	⚪ Medium	class_weight='balanced' in RF	Mitigated

Tomorrow's Preview (Day 10 — Week 2 Day 4)

Per MASTER_DOC_PART5 §3 Week 3 prep:

- Autoencoder (TensorFlow/Keras) full implementation
- Run `train_all.py` on VPS with actual NSL-KDD data
- Ensemble scorer implementation
- Evaluate all 3 models — document metrics for academic submission
- Begin ML Worker (Redis subscriber → inference loop)

Deferred Tasks (Full-Stack Dev)

Tracked in `docs/FRONTEND_TASKS_DAY8.md`:

Task	Priority	Status
All 10 frontend tasks	● - ○	Assigned to Full-Stack Dev

Reference Documents

Document	Section	Purpose
MASTER_DOC_PART4	§2.1, §2.4	NSL-KDD dataset spec + preprocessing
MASTER_DOC_PART4	§3.1-3.2	Feature engineering
MASTER_DOC_PART4	§4.1-4.5	Isolation Forest spec
MASTER_DOC_PART4	§5.1-5.5	Random Forest spec
MASTER_DOC_PART4	§7	Model evaluation framework
MASTER_DOC_PART4	§1.2	Ensemble scoring weights
MASTER_DOC_PART5	§2.1	ML directory structure
MASTER_DOC_PART5	§3	Week 2-3 plan
FRONTEND_TASKS_DAY8.md	Full doc	Deferred frontend tasks

Task workflow for Day 9 (Week 2 Day 3) — ThreatMatrix AI Sprint 2
Focus: NSL-KDD Validation + Isolation Forest + Random Forest + Evaluation Framework
Owner: Lead Architect — Frontend deferred to Full-Stack Dev